

2o-Parcial-2025.pdf



El_mensajero



Concurrencia



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid**



[Accede al documento original](#)

Concurrencia (2º parcial) (Solución)

Curso 2024–2025 - 2º semestre (junio 2025)

Grado en Ingeniería Informática / Grado en Matemáticas e Informática / 2ble. grado en Ing. Informática y ADE

UNIVERSIDAD POLITÉCNICA DE MADRID

NORMAS: Este es un cuestionario que consta de 9 preguntas. Todas las preguntas son de respuesta simple excepto la pregunta 9 que es una pregunta de desarrollo. La puntuación total del examen es de 10 puntos. La duración total es de **una hora. Sólo hay una respuesta válida a cada pregunta de respuesta simple.** Toda pregunta en que se marque más de una respuesta se considerará incorrectamente contestada. Las respuestas deben consignarse en el casillero de la derecha. Toda **pregunta incorrectamente contestada restará** del examen una cantidad de puntos igual a la puntuación de la pregunta dividida por el número de alternativas ofrecidas en la misma. No olvidéis **rellenar vuestros datos.**

RESPUESTAS

1	2	3	4
5	6	7	8

Cuestionario

- (1 punto) 1. Considere el siguiente fragmento de código, donde se han omitido las conversiones de tipos (*castings*) para hacerlo más legible:

<pre>class P1 implements CProcess{ public void run(){ String s, w; s = "A"; ch1.out().write(s); System.out.print("AS"); w = ch3.in().read(); } }</pre>	<pre>class P2 implements CProcess{ public void run(){ String s; System.out.print("C"); s = ch1.in().read(); ch2.out().write(s); } }</pre>	<pre>class P3 implements CProcess{ public void run(){ String s, w; w = "I"; s = "I"; s = ch2.in().read(); ch3.out().write(w); System.out.print(s); } }</pre>
<pre>Any2OneChannel ch1 = Channel.any2one(); Any2OneChannel ch2 = Channel.any2one(); Any2OneChannel ch3 = Channel.any2one(); public static final void main(final String[] args){ new Parallel (new CProcess[] {new P1(), new P2(), new P3()}).run(); }</pre>		

Se pide señalar la respuesta correcta.

- (a) ☐ La salida del programa siempre será CASI.
 (b) ☒ Una salida del programa podría ser CASA.
 (c) ☐ El programa se podría quedar bloqueado y no siempre acabaría.

- (1 punto) 2. Con la misma declaración de canales y mismo main, se tiene esta otra implementación de las tareas:

<pre>class P1 implements CProcess{ public void run(){ while (true) { ch1.out().write(null); ch2.in().read(); } } }</pre>	<pre>class P2 implements CProcess{ public void run(){ while (true) { ch1.in().read(); ch2.out().write(null); ch1.in().read(); ch3.out().write(null); } } }</pre>	<pre>class P3 implements CProcess{ public void run(){ while (true) { ch1.out().write(null); ch3.in().read(); } } }</pre>
--	--	--

Se pide señalar la respuesta correcta:

- (a) ☐ El programa funcionará continuamente sin fallos.
 (b) ☒ El programa puede sufrir de interbloqueos.
 (c) ☐ El programa no compilará pues no se puede pasar null por los canales.

- (1 punto) 3. Se va a realizar una implementación con monitores y nuestra metodología de una operación *op* de un CTAD cuya CPRE depende del estado del recurso y de un parámetro *c* que puede tomar los valores LEFT, AHEAD, RIGHT. Sabemos que *op* va a ser llamada por exactamente 5 procesos.

Se pide señalar la respuesta correcta:

- (a) ☒ Es suficiente con usar 3 *conditions*.
 (b) ☐ Bastaría una sola *condition*.
 (c) ☐ Son necesarias 15 *conditions*.

- (1 punto) 4. Supongamos que tenemos un objeto servicios de clase *Alternative* creado a partir de un array de *inputs* {ch0.in(), ch1.in(), ch2.in()} y un array sincCond de tres booleanos {cond0, cond1, cond2}.

En un momento de la ejecución del programa la llamada a servicios.fairSelect(sincCond) devuelve el valor 2. **Selecciona** la respuesta que define la situación que se cumple en ese momento:

- (a) ☐ cond2 es cierta y con seguridad cond0 y cond1 son falsas.
 (b) ☒ cond2 es cierta y hay un mensaje esperando en ch2.
 (c) ☐ Hay un mensaje esperando en ch2 y no hay mensajes esperando en ch0 ni en ch1.

- (1 punto) 5. Asumimos la misma declaración de la pregunta anterior respecto a servicios.

Si se llama a servicios.fairSelect(sincCond) y en ese momento cond0 y cond2 son False y cond1 es True y hay un mensaje disponible en ch1 y no hay mensajes disponibles ni en ch0 ni en ch2, ¿cómo se comportará?

Selecciona la respuesta correcta:

- (a) ☐ La ejecución de servicios.fairSelect(sincCond) no hace nada (equivalente a una instrucción vacía) y pasa a la siguiente instrucción.
 (b) ☒ La ejecución espera a que haya un mensaje listo para leer en ch0 o en ch2.
 (c) ☐ La ejecución espera a que haya mensajes listos para leer tanto en ch0 como en ch2.

- (1 punto) 6. En la implementación con monitores de un recurso se tienen tres operaciones *op1*, *op2*, *op3*. Las operaciones se han programado con este esquema de código:

<pre>public class M { private Monitor mutex = new Monitor(); private Monitor.Cond cond1 = mutex.newCond(); private Monitor.Cond cond2 = mutex.newCond();</pre>		
<pre>public void op1 () { mutex.enter(); if (!C1) cond1.await(); //Aqui se debe cumplir C1 ... mutex.leave(); }</pre>	<pre>public void op2 () { mutex.enter(); if (!C2) cond2.await(); //Aqui se debe cumplir C2 ... mutex.leave(); }</pre>	<pre>public void op3 () { mutex.enter(); ... if (C1) { cond1.signal(); } else if (C2) { cond2.signal(); } mutex.leave(); }</pre>

Señalar el comportamiento de este código:

- (a) ☐ Es un código correcto.
 (b) ☒ No debe escribirse ese código porque pueden provocarse esperas innecesarias.
 (c) ☐ No debe escribirse ese código porque pueden ejecutarse operaciones sin cumplirse su CPRE.

- (1 punto) 7. Se define una clase *servidor* P y dos clases *cliente* P1 y P2 que se comunican a través de los canales de comunicación petA y petB (se muestran las partes relevantes del código para este problema).

```

class P implements CProcess {
    public void run() {
        int n = 0;

        boolean p = true;
        boolean q = false;

        final int A = 0;
        final int B = 1;

        final Guard[] entradas =
            {petA.in(), petB.in()};
        final Alternative servicios =
            new Alternative (entradas);
        final boolean[] sincCond =
            new boolean[2];

        while (true) {
            sincCond[A] = p;
            sincCond[B] = q;

            int sel = servicios.fairSelect(sincCond);
            switch (sel) {
                case A:
                    petA.in().read();
                    n++;
                    p = !p;
                    q = !q;
                    break;
                case B:
                    petB.in().read();
                    n++;
                    q = ((n % 2) != 0);
                    break;
            }
        }
    }
}

```

```

class P1 implements CProcess {
    public void run() {
        while (true) {
            petA.out().write(null);
        }
    }
}

```

```

class P2 implements CProcess {
    public void run() {
        while (true) {
            petB.out().write(null);
        }
    }
}

```

Dado un programa concurrente con tres procesos p, p1 y p2 de las clases P, P1 y P2.

Se pide marcar cuál de las siguientes afirmaciones es la correcta:

- (a) ☒ Es seguro que los tres procesos se van a bloquear.
- (b) ☐ Es seguro que p1 acabará bloqueándose, pero p y p2 podrían seguir ejecutando indefinidamente.
- (c) ☐ Es posible que p2 se bloquee pero en ese caso p y p1 podrían seguir ejecutando indefinidamente.
- (d) ☐ Ninguna de las otras respuestas.

- (1 punto) 8. El array **boolean** sincCond [] en la implementación anterior nos sirve para:

Se pide señalar la respuesta correcta.

- (a) ☐ Almacenar en qué canales tenemos peticiones esperando para indicar a fairSelect qué canales puede seleccionar.
- (b) ☒ Almacenar la evaluación de cada una de las CPREs para indicar a fairSelect qué canales puede seleccionar.

- (2 puntos) 9. A continuación os proporcionamos la especificación de un recurso compartido que controla el acceso a una conocida estación de carga eléctrica:

C-TAD ZonaRecarga

OPERACIONES

ACCIÓN entrar: TipoVehiculo[e]

ACCIÓN salir: TipoVehiculo[e]

SEMÁNTICA

TIPO TipoVehiculo = A | B

TIPO ZonaRecarga = $\mathbb{N}$

INICIAL: self = 0

INVARIANTE: $0 \leq \text{self} \leq \text{MAXPOWER}$

CPRE: $(v = A \Rightarrow \text{self} + 50 \leq \text{MAXPOWER}) \wedge (v = B \Rightarrow \text{self} + 120 \leq \text{MAXPOWER})$
 entrar(v)

POST: $(v = A \Rightarrow \text{self} = \text{self}^{\text{pre}} + 50) \wedge (v = B \Rightarrow \text{self} = \text{self}^{\text{pre}} + 120)$

CPRE: Cierto

salir(v)

POST: $(v = A \Rightarrow \text{self} = \text{self}^{\text{pre}} - 50) \wedge (v = B \Rightarrow \text{self} = \text{self}^{\text{pre}} - 120)$

Este recurso limita el acceso para que la potencia de carga de todos los vehículos en la zona de recarga no supere MAXPOWER (20000 Kw). Se consideran dos tipos de vehículo: los de clase A que cargan a 50 Kw, y los de clase B, que cargan a 120 Kw.

Se pide completar la implementación de este recurso mediante monitores. Podéis optar tanto por un método común de desbloqueo como por código diferente en cada uno de los métodos de la interfaz ZonaRecarga. (Siguiente página.)

Para implementar este recurso con monitores hemos de tener en cuenta que hay una operación no bloqueante (*salir(t)*) y otra bloqueante (*entrar(t)*) cuya CPRE depende del parámetro *t*. Puesto que este parámetro sólo puede tomar dos valores, lo más sencillo es usar indexación de parámetros, lo cual lleva a tener una *condition* para cada caso.

Tenéis una posible implementación en la siguiente página.

El error más frecuente es intentar resolverlo con una sola condition. Este tipo de soluciones pueden puntuar un máximo de 1 punto, si bien en la mayoría de ocasiones esto viene acompañado de algún otro problema motivado por la imposibilidad de acceder al tipo de vehículo al que se va a desbloquear.

Luego, algunas de las soluciones que han llevado a cabo el esquema de indexación de parámetros pueden sufrir de otros problemas, tales como:

- Doble signal
- Esperas innecesarias por realizar forzar el signal en una condition sin comprobar que hay procesos bloqueados en ella
- Desbloques inseguros (no comprobación de CPRE en signal)
- Esperas innecesarias por no llamar a *desbloqueo()* desde *entrar(t)*, etc.

Generalmente estos fallos se castigan con medio punto.

```
import es.upm.babel.cclib.Monitor;

public class ZonaRecargaMonitor implements ZonaRecarga {

    static final int MAXPOWER = 20000; // limite total de potencia de carga
    static final int A = 0;           // tipos de vehiculo por potencia de carga
    static final int B = 1;
    static final int[] POW = {50, 120};

    private int self;                // potencia enchufada

    // Declaracion de variables para sincronizacion
    Monitor mutex;
    Monitor.Cond[] c_entrar = new Monitor.Cond[2];

    public ZonaRecargaMonitor() {
        self = 0;
        mutex = new Monitor();
        c_entrar[A] = mutex.newCond();
        c_entrar[B] = mutex.newCond();
    }

    public void entrar(int tipo) {
        mutex.enter();
        // comprobacion CPRE / bloqueos
        if (self + POW[tipo] > MAXPOWER)
            c_entrar[tipo].await();

        // establecimiento de la POST
        self += POW[tipo];

        // desbloqueo
        desbloqueo();

        mutex.leave();
    }

    public void salir(int tipo) {
        mutex.enter();
        // comprobacion CPRE / bloqueos

        // establecimiento de la POST
        self -= POW[tipo];

        // desbloqueo
        desbloqueo();

        mutex.leave();
    }

    private void desbloqueo() {
        // usad este codigo si no habeis programado desbloques especificos
        // en entrar y salir

        // damos cierta preferencia a los coches de mayor potencia de carga
        if (self + POW[B] <= MAXPOWER && c_entrar[B].waiting() > 0)
            c_entrar[B].signal();
        else if (self + POW[A] <= MAXPOWER) // ultimo caso
            c_entrar[A].signal();
    }
}
```